

Part 0

Start here: one model, two games

How the same Blender train that goes into Train Sim Classic also becomes a Transport Fever 3 vehicle mod, where the two pipelines part company, and what each game needs that the other does not.

The TF3 guide set

Part	Title	What you get from it
0	Start Here	The branch point, the two pipelines side by side, the TF3 mod folder, conventions, glossary.
1	What a TF3 Vehicle Actually Is	The .mdl model tree, metadata structs, node names as the contract, events instead of control values.
2	Preparing the Shared Model for TF3	Axes, origins, node and material naming, LOD files, textures (PBR), what to drop and what to add.
3	Animations for TF3	Event names, NLA strips, wheel/door/forever animations, direction-dependent parts, pantographs.
4	Model Editor and Metadata	FBX import, the metadata structs field by field, validation, icons, multiple units, buy groups.
5	Scripting in TF3	Transformators, what moves from TSC Lua and what does not, the trolley-pole case.
6	Build, Test and Publish	Staging area, iteration loop, in-game test order, mod.io, troubleshooting.
7	The Branch Tooling	TF3 Export Prep add-on, naming map, texture conversion, further script ideas.

Sources: the Transport Fever 3 Wiki modding manual pages supplied with this project (Model Editor, Vehicle Basics, Vehicle Types, Vehicle Advanced Topics, Repaint Mods, API index) and the TSC guide set in [guide/](#). Statements taken from those pages are marked **manual**; statements inferred from Transport Fever 2 practice or needing confirmation in your installation are marked **check**.

Where the pipelines branch

Both games want the same thing from Blender: a clean, correctly scaled, well-named model with its moving parts keyed. After that they diverge completely. Train Sim Classic wants an `.igs` plus XML blueprints and a per-vehicle Lua script; Transport Fever 3 wants **FBX files, one per level of detail**, imported by its own **Model Editor**, which writes the `.mdl` / `.msh` / `.mtl` files and holds the vehicle's metadata. Nothing from the TSC blueprint side is reusable in TF3, and nothing from the TF3 metadata side is reusable in TSC. The model and its animations are.

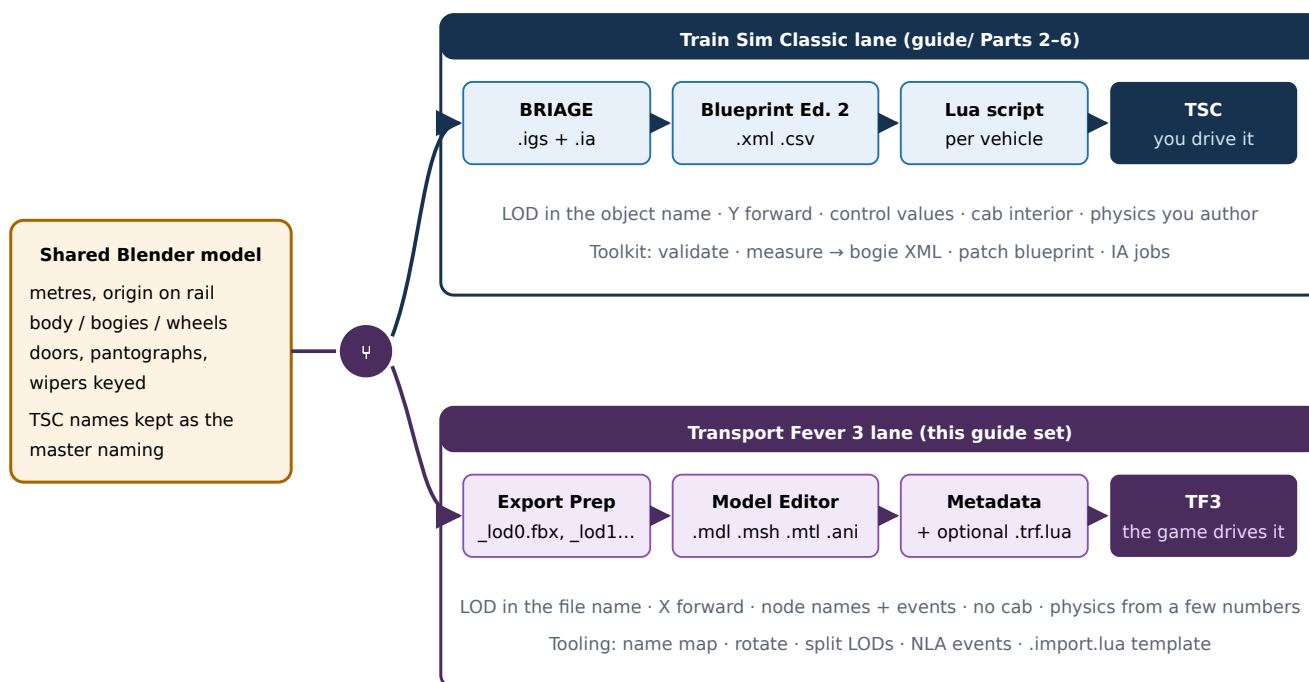


Figure 0.1 · The branch. Everything left of the fork is shared and stays named the TSC way (that naming is stricter, so it is the better master). The TF3 Export Prep add-on (Part 7) performs the renaming, rotation and LOD split on a temporary copy, so the shared scene is never edited for TF3's sake.

What is the same and what is different

Topic	Train Sim Classic	Transport Fever 3
Who drives	The player, from a modelled cab, through control values.	The game's simulation. No cab interior, no driver controls, no HUD.
Model file	One <code>.igs</code> with all LODs; cab is a second <code>.igs</code> .	One FBX per LOD (<code>name_lod0.fbx</code> , <code>_lod1</code> ...), converted to <code>.msh</code> per node by the Model Editor. manual
Where LOD lives	In the object name: <code>1_0500_engine</code> .	In the file name; the node is just <code>body</code> , suffixed <code>_lod0</code> automatically. manual
Forward axis in Blender	+Y.	+X (metadata positions “along the lane” are x; seats sit at $x = \pm 8.5$ on a loco). manual

Topic	Train Sim Classic	Transport Fever 3
Up axis	Z (Blender) → Y in the file; BRIAGE swaps.	Z; export FBX with Z up, tick “Use Blender Coordinate System” on import. manual
How parts are identified	Node names inside the IGS, referenced from blueprints.	Node names inside the .mdl , referenced from metadata (axles = { "w1", "w2" }). manual
Bogies and wheels	Bogie blueprint per bogie; axle nodes, radius, offsets typed into XML.	Any node containing axle meshes is a bogie; its origin is the pivot; axles rotate about their own y axis. Nothing to type except the axle list. manual
Animation	One .ia per animation, played by a control value (scrubbed) or toggled.	Keyframes inside the FBX become .ani files; each is named after a game event (wheels , open_doors_left , forward_parts_on ...) that the game fires. manual
Materials	Shader name + texture slots (diffuse, specular...).	PBR: albedo(+opacity), normal, metal/gloss/AO, colour-blend/dirt/rust, emissive. Material type chosen by shortcode . manual
Physics	Full engine simulation blueprint: curves, brakes, air system.	Four numbers per engine type: power kW, tractive effort kN, top speed m/s, weight kg (+ friction, brake deceleration). manual
Scripting	Per-vehicle Lua reading/writing controls every frame.	Optional transformator scripts that only compute animation states and particles; no gameplay logic per vehicle. manual
Passengers / cargo	Not modelled.	Compartments, capacities, seat locators, load indicators; the game shows people and cargo. manual
Numbers / liveries	Digit decals + numbering CSV.	Labels (text locators) and colour-blend masks; repaint mods are a formal thing. manual
Testing tool	BPE2 preview, then the game.	Model Editor: live reload on file change, validation button, screenshots/icons. manual
Distribution	Assets folder / .rwp.	Mod folder with mod.json , modinfo.json ; shared through mod.io. manual

What the supplied manual covers, and what it does not

Covered (49 pages) - Model Editor: UI, loading/saving, FBX import rules, naming of nodes/materials/textures, animations via NLA strips, lights, seats, emitters, bounding box, labels, bulk import, validation messages, icons, options. - Vehicle Basics: emissions, <code>transportVehicle</code> , compartments, seats, sound. - Vehicle Types: <code>landVehicle</code> , road/rail/water/air structs, animation events, multiple units. - Advanced: transformator scripts, reversible trains and direction-dependent parts, buy-menu groups, fake bogies. - Repaint mods; API index.	Not in the supplied pages check Mod Definition (<code>mod.json</code> , <code>modinfo.json</code> fields), Resource Types & Structure, the <code>.mdl</code> / <code>.msh</code> / <code>.mtl</code> reference pages, Guidelines & Requirements (polygon and texture budgets), Best Practices, Publish a mod, External Tools. - The material shortcode table the import names rely on. - Where this guide needs those, it states what is known from the covered pages and from Transport Fever 2 practice, and marks it check . Fetch those wiki pages before your first real import.
--	--

The TF3 mod folder

<userdata>/mods/ (or the staging area the Model Editor saves to)	
<code>myname_myloco_1/</code>	← mod id (what other mods reference as modid::)
<code>mod.json</code>	dependencies, parameters (check)
<code>modinfo.json</code>	name, authors, description, preview image for the mod browser
<code>strings.json</code>	localised VEHICLE_..._NAME / _DESCRIPTION
<code>content/</code>	
<code>vehicle/train/myloco/</code>	from import folder “vehicle-train-myloco”
<code>myloco.mdl</code>	Lua table: lods → node tree → mesh + materials; metadata
<code>msh/body_lod0.msh w1_lod0.msh ...</code>	one per node per LOD (+ <code>.msh.blob</code>)
<code>mat/body.mtl ... ani/wheels.ani ...</code>	materials; animations named by event
<code>textures/body_alb.dds _nrm _mga _cdr</code>	.tga converted to .dds on save
<code>ui/models_small/... ui/models_large/...</code>	icons from the SCREENSHOT button

Figure 0.2 · A TF3 vehicle mod. Folder placement under `content/` follows the import folder name (`<folder1>-<folder2>-<mdlname>/` → `folder1/folder2/mdlname/mdlname.mdl`) [manual](#) . Exact sub-folder names (`msh` , `mat` , `ani` , `textures` , `ui`) are the TF2 convention and the ones the manual's examples show [check](#) .

Conventions in this set

Convention	Meaning
“TSC guide”	The eight PDFs in <code>guide/</code> . Shared modelling rules are not repeated here; this set only states what TF3 adds or changes.

Convention	Meaning
TF3 vehicle axes	X = forward (along the lane), Y = left, Z = up, metres, origin on the rail head at the vehicle centre. manual for X-forward and “origin of an axle centred above the lane”; the rest check .
<code>body</code> , <code>w1</code> , <code>bogie_front</code>	TF3 node names as the Export Prep add-on generates them from <code>1_0500_engine</code> , <code>1_0500_bo01wh01</code> , <code>1_0500_bo01</code> .
manual	Stated in the supplied TF3 wiki pages.
check	Inferred or from TF2; confirm in your Model Editor / game.

Glossary

Term	Meaning
Model Editor	Urban Games' tool (<code>ModelEditor.bat/.sh/.command</code>) that imports FBX, edits metadata and materials, validates, and makes icons. manual
<code>.mdl</code>	A Lua file returning the model: <code>lods</code> (node trees with mesh and material references) and <code>metadata</code> (vehicle structs).
<code>.msh</code> / <code>.msh.blob</code>	One mesh per node per LOD, written by the Model Editor from the FBX. manual
<code>.mtl</code>	Material: type plus texture map references (<code>map_albedo_opacity</code> , <code>map_normal</code> , <code>map_metal_gloss_ao</code> , <code>map_cblend_dirt_rust</code> ...). manual
<code>.ani</code>	Animation generated from FBX keyframes; named after the event. manual
Event	A named moment the game triggers on a vehicle (<code>open_doors_left</code> , <code>brake_lights_on</code> ...) or a continuous channel (<code>wheels</code> , <code>drive</code> , <code>forever</code>). manual
Transformer	A <code>.trf.lua</code> + script that turns simulation state into animation states and particle settings. manual
Fake bogie	A scripted pivot point on a node that has no real bogie (articulated vehicles). manual
Multiple unit	A fixed consist of several <code>.mdl</code> s defined in a <code>.mu.lua</code> . manual
Staging area	Mods folder the Model Editor reads and writes; found via the game's user data folder. manual
<code>mod.io</code>	The platform TF3 mods are shared through. manual

How to use this set

Read Part 1 to understand what TF3 wants, then Part 2 before you touch the shared model: three habits (origins on hinges, separate axle objects, a driver empty) cost nothing in TSC and save a day in TF3. Use Part 7's Export Prep for every TF3 export so the shared scene never drifts.